

DECODING-VELOCITY REWARD: A PRINCIPLED PER-TOKEN SIGNAL FOR GRPO

Fangyuan Yu
Thoughtworks

Shuang Wu
Thoughtworks

Runyan Tan
Thoughtworks

Phillip Howard
Thoughtworks

ABSTRACT

Group Relative Policy Optimization (GRPO) has become the standard recipe for reinforcement-learning fine-tuning of reasoning language models, yet it inherits four interlocking weaknesses: long, redundant chains of thought; the absence of a principled per-token reward; a learning-signal collapse when the base model has near-zero pass@ K ; and a tendency to indiscriminately penalise every token of a failed trajectory. We propose the *decoding-velocity reward*, a dense per-token signal derived via potential-based reward shaping from the change in answer-decoding likelihood. The trajectory-level sum of velocities telescopes *exactly* into the *information gain* contributed by the chain of thought, $\log p_\theta(a \mid q, o) - \log p_\theta(a \mid q)$, giving every emitted token an explicit incentive to advance decoding of the answer. We argue that this formulation directly mitigates redundancy, lifts the per-token signal, and rehabilitates failed trajectories without inducing mode collapse.

1 INTRODUCTION

Reinforcement learning with verifiable rewards has driven recent progress on reasoning-heavy benchmarks, with Group Relative Policy Optimization (GRPO) (Shao et al., 2024; DeepSeek-AI, 2025) emerging as the de-facto algorithm. Despite its empirical success, the algorithm carries several structural flaws that become increasingly visible as chain-of-thought (CoT) sequences grow longer and tasks grow harder. We identify four such drawbacks (Section 2) and propose a single, mathematically clean modification of the reward (Section 3) that addresses three of them outright and lays the groundwork for tackling the fourth.

2 THE FOUR DRAWBACKS OF GRPO

(D1) CoT redundancy. Reasoning models routinely emit thousands of CoT tokens for problems that, in principle, require only a few. Much of the chain is filler that neither constrains the answer distribution nor verifies intermediate computation. Practitioners have addressed this with explicit length penalties or short-answer bonuses, but such auxiliary rewards are fundamentally unprincipled: a hand-tuned length term is a separate objective bolted onto the correctness reward and almost always trades against accuracy, forcing brittle coefficient sweeps to balance the two. What is missing is a *single* reward whose structure already prices the cost of additional tokens against their contribution to solving the problem.

A striking side-symptom makes the severity of redundancy concrete: under a *fixed* CoT token budget $T_{\max}$, larger models in the same family can underperform smaller ones on the same task, simply because their characteristic CoT exceeds $T_{\max}$ and is truncated before the answer is emitted. Equivalently, holding accuracy fixed, the CoT budget required grows super-linearly with model scale even when the underlying problem does not. We document this size–budget inversion empirically on Game of 24 in Section 4; it is, in effect, a scale-dependent manifestation of D1: the larger the model, the more of its emitted chain is filler whose only effect is to push the answer past the budget cliff. A principled reward should therefore tighten as model scale increases, not require a larger $T_{\max}$ to compensate.

(D2) No principled per-token signal. The reward in GRPO is sparse and trajectory-level. Every intermediate token inherits the same advantage estimate, leaving the per-token credit assignment uncalibrated and the gradient noisy.

(D3) The zero-pass@ K dead zone. When the base model has near-zero pass@ K on a task, the group of sampled trajectories contains no positives, the relative advantage collapses to zero, and the policy receives no learning signal. The model never lifts off such tasks.

(D4) Indiscriminate credit assignment. GRPO discriminates against *every* token in a failed trajectory and accepts *every* token in a successful one, regardless of whether that token was actually responsible for the outcome. This invites mode collapse, discards genuinely useful sub-trajectories that happened to terminate badly, reinforces filler tokens that happened to ride along with success, and is a poor model of how credit propagates through reasoning.

3 METHOD: DECODING-VELOCITY REWARD

3.1 SETUP AND NOTATION

Let q denote a query, $o = (o_1, o_2, \dots, o_T)$ a chain-of-thought sequence generated by the policy, and a the answer. The policy is the language model p_θ . We assume access to a reference answer a per query (e.g. a gold answer; the formulation extends to a set of equivalent answers by taking an expectation).

3.2 SEQUENCE-LEVEL: INFORMATION-GAIN REWARD

We define the sequence-level CoT reward as the *information gain* of the answer under the policy when the CoT is conditioned on, relative to the direct-answer baseline:

$$r(a, o, q) := \log p_\theta(a \mid q, o) - \log p_\theta(a \mid q). \quad (1)$$

Adopting the convention $r(a, q) := \log p_\theta(a \mid q)$ for the no-CoT case, $r(a, o, q)$ measures *how much the CoT helps* the policy decode the answer. This is a more principled target than the raw decoding likelihood $\log p_\theta(a \mid q, o)$: the latter is satisfied by any prefix that merely *correlates* with a , whereas Equation 1 rewards the CoT’s *causal contribution* (Section 3.7).

3.3 TOKEN-LEVEL: VELOCITY REWARD

For each token o_t , we define the *velocity reward* as the change in sequence-level reward induced by emitting that token:

$$v(a, o_{1:t}, q) := r(a, o_{1:t}, q) - r(a, o_{1:t-1}, q) = \log p_\theta(a \mid q, o_{1:t}) - \log p_\theta(a \mid q, o_{1:t-1}). \quad (2)$$

By telescoping, the trajectory return equals the sequence-level reward exactly:

$$\sum_{t=1}^T v(a, o_{1:t}, q) = \sum_{t=1}^T [r(a, o_{1:t}, q) - r(a, o_{1:t-1}, q)] = r(a, o, q). \quad (3)$$

Equivalence. Equation 3 establishes that *optimising the per-token velocity reward is exactly equivalent, at the trajectory level, to optimising the information gain contributed by the CoT*. No bound, no approximation, no shaping coefficient.

3.4 ONLINE ANSWER BUFFER

The formulation so far conditions on a single reference answer a , but many queries admit *multiple* correct answers – numerically equivalent expressions, equivalent code, or paraphrastic restatements. Optimising against a single a risks overfitting the CoT to a specific surface form of the correct answer rather than to the underlying solution.

We therefore maintain an *online answer buffer* $\mathcal{A}_q$ per query, populated by all correct answers the policy has produced so far, as judged by the verifiable reward function:

$$\mathcal{A}_q := \{a \mid a \text{ is a correct answer for } q\}. \quad (4)$$

At training time we sample $a \sim \text{unif}(\mathcal{A}_q)$ when evaluating the information-gain reward, replacing $\log p_\theta(a \mid q, o)$ in Equations 1 and 2 with $\mathbb{E}_{a \sim \text{unif}(\mathcal{A}_q)}[\log p_\theta(a \mid q, o)]$. This encourages the policy to learn a *general* CoT that decodes the class of correct answers, rather than overfitting to any one of them. The buffer is built online from the model’s own successful rollouts and grows monotonically over training; until a query has at least one entry in $\mathcal{A}_q$, we fall back to the gold reference answer.

3.5 ONLINE PREFIX BUFFER

Symmetric to the answer buffer of Section 3.4, we maintain a *prefix buffer* $\mathcal{P}_q$ per query, recording CoT prefixes the model has produced so far that score highly under the decoding objective *relative to other prefixes of the same length*:

$$\mathcal{P}_q := \{o_{1:t} \mid 1 \leq t < T\}, \quad (5)$$

organised as a priority queue. Concretely, we bin prefixes by length t and, within each bin, retain the top- K prefixes ranked by the cumulative velocity reward $\sum_{s \leq t} v(a, o_{1:s}, q) = \log p_\theta(a \mid q, o_{1:t}) - \log p_\theta(a \mid q)$ – i.e. the partial information gain achieved by the prefix.

During rollout, we occasionally seed the policy with a sampled prefix from $\mathcal{P}_q$ and let it continue from there. This serves two purposes: (i) it amortises the cost of rediscovering productive CoT openings, and (ii) it injects exploration around *known-good* subsequences without discarding the benefits of unconstrained sampling. Together with the answer buffer, the two structures form a complementary memory: $\mathcal{A}_q$ diversifies the *target* the CoT is graded against, $\mathcal{P}_q$ diversifies the *starting point* from which the CoT is built.

3.6 PER-TOKEN ADVANTAGE

GRPO computes a single advantage per trajectory and broadcasts it to every token. With a genuinely per-token reward in hand, we compute a per-token advantage instead. The challenge is that rollouts in a group have different CoT lengths, so per-position statistics are not directly comparable.

Consider a group of N rollouts with per-token velocity rewards

$$\{(o_{1:T_i}^i, v_{1:T_i}^i)\}_{i=1}^N, \quad v_t^i := v(a, o_{1:t}^i, q).$$

We truncate the CoT sequence of every rollout to a common length T^* defined as *the shortest length among rollouts that achieve the highest trajectory-level information gain*:

$$T^* := \min \left\{ T_i \mid i \in \arg \max_j \sum_{t=1}^{T_j} v_t^j \right\}. \quad (6)$$

After truncation, every rollout’s CoT has length T^* and per-position advantages can be computed by standard group-relative normalisation across the N rollouts at each $t \in \{1, \dots, T^*\}$.

Why this rule. The truncation policy explicitly favours the *shortest* winner: among all trajectories that achieve the maximum trajectory return, it selects the most concise. Rollouts longer than T^* have their tail tokens dropped from the loss, so a longer trajectory is preferred only if it strictly improves the trajectory return. This realises the “length and accuracy in one signal” property of D1 at the level of credit assignment: redundant tokens past T^* receive neither reward nor advantage, and shorter, denser CoTs become the implicit target of optimisation.

3.7 WHAT THIS BUYS US

D1 – CoT redundancy is priced at every scale. Redundancy is captured at *both* granularities by the same equation. A redundant CoT contributes zero information gain over the direct-answer baseline, since $\log p_\theta(a \mid q, o) \approx \log p_\theta(a \mid q)$ makes $r(a, o, q) \approx 0$. Symmetrically, a redundant token contributes zero *velocity*, since $\log p_\theta(a \mid q, o_{1:t}) \approx \log p_\theta(a \mid q, o_{1:t-1})$ makes $v(a, o_{1:t}, q) \approx 0$. The reward thus aligns with the predictive target at the sequence level and prices each token by its *marginal* contribution to that target – without an auxiliary length penalty, and without any coefficient to tune.

D2 – The per-token signal is exactly the sequence-level reward. By Equation 3, assigning the per-token reward $v(a, o_{1:t}, q)$ is *exactly* equivalent, in trajectory return, to assigning the sequence-level reward $r(a, o, q)$. The per-token signal is therefore not a heuristic surrogate but a strict decomposition of the sequence-level objective: principled in form and aligned in content with what we ultimately optimise.

D3 – Toward learning, not just sharpening (hypothesis). We hypothesise – and intend to validate experimentally – that the dense per-token velocity signal helps the model *learn new behaviours* rather than merely *sharpen* the distribution it already places mass on. Vanilla GRPO can only reweight existing trajectories; in the zero-pass@ K regime it has nothing to reweight. The velocity reward, by contrast, scores *every* token of *every* rollout against the reference answer, so non-zero gradients exist even when the group contains no successful trajectory. Whether this is sufficient to lift the model off zero-pass@ K tasks is an open empirical question that we leave to Section 4.

D4 – Credit follows contribution, not outcome. Failed trajectories contain tokens whose local velocity is positive – locally, they advanced decoding of the reference answer – and these are rewarded. Successful trajectories contain tokens whose local velocity is zero or negative – they did not contribute, or actively dragged the decoding likelihood backward – and these are discouraged. Credit attaches to a token by its *measured contribution*, not by the eventual outcome of the trajectory it happens to belong to. This eliminates the “discriminate every token of a failure, accept every token of a success” pathology of vanilla GRPO at its source.

4 EXPERIMENTS

This section is under active development. We outline below the experiments that will populate this section.

4.1 RESEARCH QUESTIONS

The Experiments section is organised around three head-to-head comparisons between vanilla GRPO and GRPO augmented with the decoding-velocity reward (plus the answer buffer $\mathcal{A}_q$ and prefix buffer $\mathcal{P}_q$ of Section 3). Each question targets one of the drawbacks that the method is designed to address.

(RQ1) Does the velocity reward shorten the CoT without hurting accuracy? This is the causal test for **D1**. Under a matched training budget and a fixed token cap $T_{\max}$, we compare mean CoT length and pass-rate trajectories between the two algorithms. The hypothesis is that the velocity reward, by pricing redundant tokens at zero per-token contribution, drives the emitted chain to a substantially shorter steady state while preserving (or improving) accuracy — something a hand-tuned length penalty achieves only by trading against correctness.

(RQ2) Does the velocity reward break the zero-pass@ K dead zone? This is the test for **D3**. We isolate the hardest difficulty bucket on which vanilla GRPO records pass-rate ≈ 0 for the entire training run, and ask whether the dense per-token signal — combined with the answer buffer that injects known-correct trajectories into the group — enables non-trivial learning where GRPO is structurally stuck. The hypothesis is that the velocity reward provides gradient even when no sampled trajectory is correct, because v_t is computed against a reference answer rather than against the group.

(RQ3) Does the prefix buffer increase solution diversity? This probes the answer/prefix-buffer design rather than a specific GRPO drawback. We measure the number of distinct correct expressions per puzzle emitted by the trained policy under temperature sampling, and compare against vanilla GRPO. We expect the prefix buffer $\mathcal{P}_q$, by randomly priming rollouts with previously discovered correct prefixes, to keep multiple solution modes simultaneously near the top of the logit ranking, rather than letting GRPO collapse onto a single canonical CoT per puzzle. We are explicit that an LLM cannot store K genuinely independent responses to a single prompt without something like a prefix-conditioning mechanism: $\mathcal{P}_q$ is precisely that mechanism, and RQ3 is the empirical check that it works.

4.2 SETUP

Models and tasks. *TBD.* Planned: Qwen3-1.7B / Qwen3-4B on GSM8K, with extensions to MATH and a code-reasoning benchmark.

Baselines. We organise baselines in three groups, each isolating a different reason the velocity reward might or might not help.

(A) *Non-RL on correct trajectories.* These are the floor: do we need RL at all, or does fine-tuning on correct rollouts suffice?

- **SFT** on a fixed pool of canonical correct solutions.
- **Rejection-sampling fine-tuning** (RFT / ReST^{EM} (Yuan et al., 2023; Singh et al., 2024)): sample, filter by verifier, supervised fine-tune, repeat.
- **STaR** (Zelikman et al., 2022): bootstrap rationales by SFT on filtered self-generated chains, with rationalisation on problems the model fails (the answer is revealed and a rationale is generated to match). This is the closest non-RL analogue to our prefix buffer $\mathcal{P}_q$.

(B) *RL with a sparse trajectory reward.* These isolate the value of moving from trajectory-level to per-token credit.

- **Vanilla GRPO** (Shao et al., 2024; DeepSeek-AI, 2025) with a verifiable correctness reward.
- **GRPO + linear length penalty:** $R \leftarrow R - \lambda|o|$. The naive variant most practitioners try first; expected to trade accuracy for length.
- **GRPO + outcome-conditional length penalty** (Kimi-1.5 style (Kimi Team, 2025)): penalise length only on correct rollouts. The strongest hand-tuned baseline for D1; the one that could plausibly match the velocity reward’s length–accuracy frontier.

For both penalised variants we report a sweep over λ and plot the resulting (length, accuracy) Pareto curve, against which the velocity-reward result is a single point.

(C) *RL with a dense per-token reward.* These are the closest competitors and isolate whether *velocity-derived* density (change in answer likelihood) is the right kind, as opposed to other analytic or learned per-token signals.

- **PRIME** (Cui et al., 2025): train an implicit process reward model with a DPO-style objective on outcome labels; its per-token log-ratio $\beta[\log \pi_\phi(o_t | \cdot) - \log \pi_{\text{ref}}(o_t | \cdot)]$ is used as the dense RL signal. Like our velocity reward, the per-token decomposition is analytic; unlike ours, the signal measures policy-vs-reference deviation rather than change in the answer posterior.
- **PRM-guided PPO** (Lightman et al., 2024; Wang et al., 2024): a learned step-level verifier supplies dense reward. Different mechanism (learned vs. analytic), same goal as PRIME and ours.

Implementation of r and v . *TBD.* We will detail how the reference answer a (or set of equivalent answers) is sourced, how $\log p_\theta(a | q, o_{1:t})$ is evaluated (single masked forward pass versus per-token recomputation), and how the velocity signal is integrated with the GRPO group-relative baseline.

4.3 MAIN RESULTS

TBD. Accuracy and CoT-length curves over training steps for baseline GRPO versus GRPO + decoding-velocity reward.

4.4 ABLATIONS

TBD. Effect of: (a) the size and diversity of the reference-answer set; (b) replacing the velocity reward with the raw decoding likelihood (to confirm the camping problem); (c) co-existence with length penalties (to verify they become redundant).

4.5 DIAGNOSTICS

TBD. Per-token reward attribution, redundancy measurements (distinct- n , self-BLEU on CoTs), and the zero-pass@ K recovery experiment.

REFERENCES

- Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Kimi Team. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2312.06585.
- Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

A APPENDIX

Reserved for derivations, hyperparameters, and additional results.